- FAPI 2.0 Security Profile

V11 Cryptography

Control Objective

The objective of this chapter is to define best practices for the general use of cryptography, as well as to instill a fundamental understanding of cryptographic principles and inspire a shift toward more resilient and modern approaches. It encourages the following:

- Implementing robust cryptographic systems that fail securely, adapt to evolving threats, and are future-proof.
- Utilizing cryptographic mechanisms that are both secure and aligned with industry best practices.
- Maintaining a secure cryptographic key management system with appropriate access controls and auditing.
- Regularly evaluating the cryptographic landscape to assess new risks and adapt algorithms accordingly.
- Discovering and managing cryptographic use cases throughout the application's lifecycle to ensure that all cryptographic assets are accounted for and secured.

In addition to outlining general principles and best practices, this document also provides more in-depth technical information about the requirements in Appendix C - Cryptography Standards. This includes algorithms and modes that are considered "approved"for the purposes of the requirements in this chapter.

Requirements that use cryptography to solve a separate problem, such as secrets management or communications security, will be in different parts of the standard.

V11.1 Cryptographic Inventory and Documentation

Applications need to be designed with strong cryptographic architecture to protect data assets according to their classification. Encrypting everything is wasteful; not encrypting anything is legally negligent. A balance must be struck, usually during architectural or high-level design, design sprints, or architectural spikes. Designing cryptography "on the fly"or retrofitting it will inevitably cost much more to implement securely than simply building it in from the start.

It is important to ensure that all cryptographic assets are regularly discovered, inventoried, and assessed. Please see the appendix for more information on how this can be done.

The need to future-proof cryptographic systems against the eventual rise of quantum computing is also critical. Post-Quantum Cryptography (PQC) refers to cryptographic algorithms designed to

remain secure against attacks by quantum computers, which are expected to break widely used algorithms such as RSA and elliptic curve cryptography (ECC).

Please see the appendix for current guidance on vetted PQC primitives and standards.

#	Description	Level
11.1.1	Verify that there is a documented policy for management of cryptographic keys and a cryptographic key lifecycle that follows a key management standard such as NIST SP 800-57. This should include ensuring that keys are not overshared (for example, with more than two entities for shared secrets and more than one entity for private keys).	2
11.1.2	Verify that a cryptographic inventory is performed, maintained, regularly updated, and includes all cryptographic keys, algorithms, and certificates used by the application. It must also document where keys can and cannot be used in the system, and the types of data that can and cannot be protected using the keys.	2
11.1.3	Verify that cryptographic discovery mechanisms are employed to identify all instances of cryptography in the system, including encryption, hashing, and signing operations.	3
11.1.4	Verify that a cryptographic inventory is maintained. This must include a documented plan that outlines the migration path to new cryptographic standards, such as post-quantum cryptography, in order to react to future threats.	3

V11.2 Secure Cryptography Implementation

This section defines the requirements for the selection, implementation, and ongoing management of core cryptographic algorithms for an application. The objective is to ensure that only robust, industry-accepted cryptographic primitives are deployed, in alignment with current standards (e.g., NIST, ISO/IEC) and best practices. Organizations must ensure that each cryptographic component is selected based on peer-reviewed evidence and practical security testing.

#	Description	Level
11.2.1	Verify that industry-validated implementations (including libraries and hardware-accelerated implementations) are used for cryptographic operations.	2

#	Description	Level
11.2.2	Verify that the application is designed with crypto agility such that random number, authenticated encryption, MAC, or hashing algorithms, key lengths, rounds, ciphers and modes can be reconfigured, upgraded, or swapped at any time, to protect against cryptographic breaks. Similarly, it must also be possible to replace keys and passwords and re-encrypt data. This will allow for seamless upgrades to post-quantum cryptography (PQC), once high-assurance implementations of approved PQC schemes or standards are widely available.	2
11.2.3	Verify that all cryptographic primitives utilize a minimum of 128-bits of security based on the algorithm, key size, and configuration. For example, a 256-bit ECC key provides roughly 128 bits of security where RSA requires a 3072-bit key to achieve 128 bits of security.	2
11.2.4	Verify that all cryptographic operations are constant-time, with no ‘short-circuit’operations in comparisons, calculations, or returns, to avoid leaking information.	3
11.2.5	Verify that all cryptographic modules fail securely, and errors are handled in a way that does not enable vulnerabilities, such as Padding Oracle attacks.	3

V11.3 Encryption Algorithms

Authenticated encryption algorithms built on AES and CHACHA20 form the backbone of modern cryptographic practice.

#	Description	Level
11.3.1	Verify that insecure block modes (e.g., ECB) and weak padding schemes (e.g., PKCS#1 v1.5) are not used.	1
11.3.2	Verify that only approved ciphers and modes such as AES with GCM are used.	1
11.3.3	Verify that encrypted data is protected against unauthorized modification preferably by using an approved authenticated encryption method or by combining an approved encryption method with an approved MAC algorithm.	2
11.3.4	Verify that nonces, initialization vectors, and other single-use numbers are not used for more than one encryption key and data-element pair. The method of generation must be appropriate for the algorithm being used.	3

#	Description	Level
11.3.5	Verify that any combination of an encryption algorithm and a MAC algorithm is operating in encrypt-then-MAC mode.	3

V11.4 Hashing and Hash-based Functions

Cryptographic hashes are used in a wide variety of cryptographic protocols, such as digital signatures, HMAC, key derivation functions (KDF), random bit generation, and password storage. The security of the cryptographic system is only as strong as the underlying hash functions used. This section outlines the requirements for using secure hash functions in cryptographic operations.

For password storage, as well as the cryptography appendix, the OWASP Password Storage Cheat Sheet will also provide useful context and guidance.

#	Description	Level
11.4.1	Verify that only approved hash functions are used for general cryptographic use cases, including digital signatures, HMAC, KDF, and random bit generation. Disallowed hash functions, such as MD5, must not be used for any cryptographic purpose.	1
11.4.2	Verify that passwords are stored using an approved, computationally intensive, key derivation function (also known as a “password hashing function”), with parameter settings configured based on current guidance. The settings should balance security and performance to make brute-force attacks sufficiently challenging for the required level of security.	2
11.4.3	Verify that hash functions used in digital signatures, as part of data authentication or data integrity are collision resistant and have appropriate bit-lengths. If collision resistance is required, the output length must be at least 256 bits. If only resistance to second pre-image attacks is required, the output length must be at least 128 bits.	2
11.4.4	Verify that the application uses approved key derivation functions with key stretching parameters when deriving secret keys from passwords. The parameters in use must balance security and performance to prevent brute-force attacks from compromising the resulting cryptographic key.	2

V11.5 Random Values

Cryptographically secure Pseudo-random Number Generation (CSPRNG) is incredibly difficult to get right. Generally, good sources of entropy within a system will be quickly depleted if over-used, but

sources with less randomness can lead to predictable keys and secrets.

#	Description	Level
11.5.1	Verify that all random numbers and strings which are intended to be non-guessable must be generated using a cryptographically secure pseudo-random number generator (CSPRNG) and have at least 128 bits of entropy. Note that UUIDs do not respect this condition.	2
11.5.2	Verify that the random number generation mechanism in use is designed to work securely, even under heavy demand.	3

V11.6 Public Key Cryptography

Public Key Cryptography will be used where it is not possible or not desirable to share a secret key between multiple parties.

As part of this, there exists a need for approved key exchange mechanisms, such as Diffie-Hellman and Elliptic Curve Diffie-Hellman (ECDH) to ensure that the cryptosystem remains secure against modern threats. The “Secure Communication”chapter provides requirements for TLS so the requirements in this section are intended for situations where Public Key Cryptography is being used in use cases other than TLS.

#	Description	Level
11.6.1	Verify that only approved cryptographic algorithms and modes of operation are used for key generation and seeding, and digital signature generation and verification. Key generation algorithms must not generate insecure keys vulnerable to known attacks, for example, RSA keys which are vulnerable to Fermat factorization.	2
11.6.2	Verify that approved cryptographic algorithms are used for key exchange (such as Diffie-Hellman) with a focus on ensuring that key exchange mechanisms use secure parameters. This will prevent attacks on the key establishment process which could lead to adversary-in-the-middle attacks or cryptographic breaks.	3

V11.7 In-Use Data Cryptography

Protecting data while it is being processed is paramount. Techniques such as full memory encryption, encryption of data in transit, and ensuring data is encrypted as quickly as possible after use is recommended.

#	Description	Level
11.7.1	Verify that full memory encryption is in use that protects sensitive data while it is in use, preventing access by unauthorized users or processes.	3
11.7.2	Verify that data minimization ensures the minimal amount of data is exposed during processing, and ensure that data is encrypted immediately after use or as soon as feasible.	3

References

For more information, see also:

- OWASP Web Security Testing Guide: Testing for Weak Cryptography
- OWASP Cryptographic Storage Cheat Sheet
- FIPS 140-3
- NIST SP 800-57

V12 Secure Communication

Control Objective

This chapter includes requirements related to the specific mechanisms that should be in place to protect data in transit, both between an end-user client and a backend service, as well as between internal and backend services.

The general concepts promoted by this chapter include:

- Ensuring that communications are encrypted externally, and ideally internally as well.
- Configuring encryption mechanisms using the latest guidance, including preferred algorithms and ciphers.
- Ensuring that communications are not being intercepted by unauthorized parties through the use of signed certificates.

In addition to outlining general principles and best practices, the ASVS also provides more in-depth technical information about cryptographic strength in Appendix C - Cryptography Standards.

V12.1 General TLS Security Guidance

This section provides initial guidance on how to secure TLS communications. Up-to-date tools should be used to review TLS configuration on an ongoing basis.

While the use of wildcard TLS certificates is not inherently insecure, a compromise of a certificate that is deployed across all owned environments (e.g., production, staging, development, and test) may lead to a compromise of the security posture of the applications using it. Proper protection, management, and the use of separate TLS certificates in different environments should be employed if possible.

#	Description	Level
12.1.1	Verify that only the latest recommended versions of the TLS protocol are enabled, such as TLS 1.2 and TLS 1.3. The latest version of the TLS protocol must be the preferred option.	1
12.1.2	Verify that only recommended cipher suites are enabled, with the strongest cipher suites set as preferred. L3 applications must only support cipher suites which provide forward secrecy.	2
12.1.3	Verify that the application validates that mTLS client certificates are trusted before using the certificate identity for authentication or authorization.	2
12.1.4	Verify that proper certification revocation, such as Online Certificate Status Protocol (OCSP) Stapling, is enabled and configured.	3
12.1.5	Verify that Encrypted Client Hello (ECH) is enabled in the application's TLS settings to prevent exposure of sensitive metadata, such as the Server Name Indication (SNI), during TLS handshake processes.	3

V12.2 HTTPS Communication with External Facing Services

Ensure all HTTP traffic to external-facing services which the application exposes is sent encrypted, with publicly trusted certificates.

#	Description	Level
12.2.1	Verify that TLS is used for all connectivity between a client and external facing, HTTP-based services, and does not fall back to insecure or unencrypted communications.	1
12.2.2	Verify that external facing services use publicly trusted TLS certificates.	1

V12.3 General Service to Service Communication Security

Server communications (both internal and external) involve more than just HTTP. Connections to and from other systems must also be secure, ideally using TLS.

#	Description	Level
12.3.1	Verify that an encrypted protocol such as TLS is used for all inbound and outbound connections to and from the application, including monitoring systems, management tools, remote access and SSH, middleware, databases, mainframes, partner systems, or external APIs. The server must not fall back to insecure or unencrypted protocols.	2
12.3.2	Verify that TLS clients validate certificates received before communicating with a TLS server.	2
12.3.3	Verify that TLS or another appropriate transport encryption mechanism used for all connectivity between internal, HTTP-based services within the application, and does not fall back to insecure or unencrypted communications.	2
12.3.4	Verify that TLS connections between internal services use trusted certificates. Where internally generated or self-signed certificates are used, the consuming service must be configured to only trust specific internal CAs and specific self-signed certificates.	2
12.3.5	Verify that services communicating internally within a system (intra-service communications) use strong authentication to ensure that each endpoint is verified. Strong authentication methods, such as TLS client authentication, must be employed to ensure identity, using public-key infrastructure and mechanisms that are resistant to replay attacks. For microservice architectures, consider using a service mesh to simplify certificate management and enhance security.	3

References

For more information, see also:

- OWASP - Transport Layer Security Cheat Sheet
- Mozilla's Server Side TLS configuration guide
- Mozilla's tool to generate known good TLS configurations.
- O-Saft - OWASP Project to validate TLS configuration

V13 Configuration

Control Objective

The application's default configuration must be secure for use on the Internet.

This chapter provides guidance on the various configurations necessary to achieve this, including those applied during development, build, and deployment.

Topics covered include preventing data leakage, securely managing communication between components, and protecting secrets.

V13.1 Configuration Documentation

This section outlines documentation requirements for how the application communicates with internal and external services, as well as techniques to prevent loss of availability due to service inaccessibility. It also addresses documentation related to secrets.

#	Description	Level
13.1.1	Verify that all communication needs for the application are documented. This must include external services which the application relies upon and cases where an end user might be able to provide an external location to which the application will then connect.	2
13.1.2	Verify that for each service the application uses, the documentation defines the maximum number of concurrent connections (e.g., connection pool limits) and how the application behaves when that limit is reached, including any fallback or recovery mechanisms, to prevent denial of service conditions.	3
13.1.3	Verify that the application documentation defines resource-management strategies for every external system or service it uses (e.g., databases, file handles, threads, HTTP connections). This should include resource-release procedures, timeout settings, failure handling, and where retry logic is implemented, specifying retry limits, delays, and back-off algorithms. For synchronous HTTP request–response operations it should mandate short timeouts and either disable retries or strictly limit retries to prevent cascading delays and resource exhaustion.	3
13.1.4	Verify that the application's documentation defines the secrets that are critical for the security of the application and a schedule for rotating them, based on the organization's threat model and business requirements.	3

V13.2 Backend Communication Configuration

Applications interact with multiple services, including APIs, databases, or other components. These may be considered internal to the application but not included in the application's standard access control mechanisms, or they may be entirely external. In either case, it is necessary to configure the application to interact securely with these components and, if required, protect that configuration.

Note: The “Secure Communication”chapter provides guidance for encryption in transit.

#	Description	Level
13.2.1	Verify that communications between backend application components that don’t support the application’s standard user session mechanism, including APIs, middleware, and data layers, are authenticated. Authentication must use individual service accounts, short-term tokens, or certificate-based authentication and not unchanging credentials such as passwords, API keys, or shared accounts with privileged access.	2
13.2.2	Verify that communications between backend application components, including local or operating system services, APIs, middleware, and data layers, are performed with accounts assigned the least necessary privileges.	2
13.2.3	Verify that if a credential has to be used for service authentication, the credential being used by the consumer is not a default credential (e.g., root/root or admin/admin).	2
13.2.4	Verify that an allowlist is used to define the external resources or systems with which the application is permitted to communicate (e.g., for outbound requests, data loads, or file access). This allowlist can be implemented at the application layer, web server, firewall, or a combination of different layers.	2
13.2.5	Verify that the web or application server is configured with an allowlist of resources or systems to which the server can send requests or load data or files from.	2
13.2.6	Verify that where the application connects to separate services, it follows the documented configuration for each connection, such as maximum parallel connections, behavior when maximum allowed connections is reached, connection timeouts, and retry strategies.	3

V13.3 Secret Management

Secret management is an essential configuration task to ensure the protection of data used in the application. Specific requirements for cryptography can be found in the “Cryptography”chapter, but this section focuses on the management and handling aspects of secrets.